

SafeSight: Driver Aid System for Blindspot Detection, Collision Warning, and Lane Departure.

Aryan Patel

May 30, 2026

Abstract

Polish this/add discussion of previous works

While modern Advanced Driver Assistance Systems (ADAS) effectively mitigate human-error accidents, previous implementations rely heavily on proprietary, high-cost sensor arrays (such as LiDAR and radar) that restrict these safety features to premium vehicles. This project addresses this accessibility gap by developing a low-cost, strictly vision-based ADAS framework engineered to democratize safety through affordable edge-computing hardware. The system merges lightweight deep learning (YOLOv8n) with classical geometric computer vision—specifically Canny Edge detection, Hough Transforms, and Exponential Moving Averages (EMA)—to create a robust perceptual safety buffer. Designed for deployment on microcomputers like the Raspberry Pi 5, the architecture executes spatial blindspot monitoring, monocular depth estimation for forward collision warnings, and dynamic lane departure tracking. Real-world testing confirmed the system’s viability, achieving a consistent 5–6 FPS processing rate with highly stabilized lane-polygon locks and accurate LiDAR-free proximity alerts, successfully proving that life-saving predictive safety can be achieved on economy hardware.

1 Introduction

Shallow right now

1. Motivation & Context (The Problem)

- The Global Issue: Start with the human cost. Briefly mention that human error (distraction, blindspots, delayed reaction) is the leading cause of traffic accidents.
- The Accessibility Gap (The “Why”): Explain that while Advanced Driver Assistance Systems (ADAS) solve this problem, they are locked behind a paywall. Emphasize the millions of older vehicles on the road that lack these basic safety features.
- our Mission: To be able to retrofit older vehicles with new driver aid systems for safety at a low cost

2. Existing Solutions & Their Limitations

- **Current High-End Tech:** Describe how modern factory ADAS works (e.g., Tesla, high-end Mercedes). Mention their reliance on sensor fusion.
- **The Hardware Limitations:** Explicitly name the proprietary sensors they use: LiDAR arrays, millimeter-wave (mmWave) radar, and high-end AI compute units (like NVIDIA Drive).
- **The Financial/Structural Limitations:** Point out that these existing solutions cost thousands of dollars, require heavy power consumption, and cannot be easily or cheaply installed into a standard 2010 sedan.

3. The SafeSight Solution (Paper Summary / Project Overview)

- **The Pivot to Vision & Edge AI:** Introduce SafeSight. Explain that you are replacing expensive sensor hardware with smart software.
- **The Core Concept:** Summarize your methodology briefly: using a standard, cheap camera feed processed locally on an affordable microcomputer (Raspberry Pi 5).
- **The Hybrid Approach:** Mention that you are combining lightweight neural networks (YOLOv8n) with classical mathematics (Hough Transforms, EMA) to make it run fast enough on low-end hardware.

4. Contributions & Final Results (What You Accomplished)

- **The New Contribution:** Clearly state what you brought to the table. (e.g., “This project contributes a highly optimized, purely vision-based perception pipeline that proves life-saving ADAS logic can be executed on a \$80 microcomputer.”)
- **Accomplished System 1 (Blindspot):** Detail that you successfully built a spatial geometric “No-Zone” mapping system achieving a 95% True Positive detection rate.
- **Accomplished System 2 (Lane Departure):** Detail that you implemented a robust Lane Departure Warning system, uniquely stabilized by an Exponential Moving Average (EMA) to prevent polygon flickering.
- **Accomplished System 3 (Collision Warning):** Detail that you engineered a LiDAR-free, monocular depth-estimation tool
- **Overall Hardware Result:** Conclude by stating the system is finished and successfully runs localized, concurrent video feeds at 5–6 FPS on the Raspberry Pi 5.

2 Procedure

1. Software & Algorithms

- **Core Libraries:** OpenCV (image processing), Ultralytics (YOLO deployment), NumPy (matrix math).
- **Deep Learning (Object Detection):** Deployed YOLOv8n (nano) for optimized edge-compute vehicle classification.
- **Classical Vision (Lane Tracking):** Utilized Canny Edge Detection and Hough Line Transforms to extract lane boundaries.

- Mathematical Filters: Applied an Exponential Moving Average (EMA) to stabilize lane coordinates across frames; used monocular pinhole camera geometry for depth estimation.

2. Experimental Design & Execution

- Pre-processing: Applied fisheye undistortion matrices (balance=0.1, scale=0.7) and dynamic lower-third camera cropping to remove vehicle dashboard glare.
- Logic Execution: Processed sequential frames to test three isolated pipelines:
- Blindspot: Point-polygon intersection testing.
- LDW: Center-screen vs. Lane-center pixel deviation.
- Collision: Bounding box width vs. assumed vehicle width (2.0m) distance scaling.
- Validation: Measured processing latency (FPS) and tracking stability against real-world driving datasets.

3. Resources & Replication

- Open-Source Weights: YOLOv8n model weights accessed via Ultralytics: [Insert Ultralytics Link]

2.1 Software and Environment

You may have SUBSECTIONS. Not required but sometimes helpful.

All source code is available at <https://github.com/yourusername/your-repo>.

Component	Detail
Language	Python 3.11.9
ML Framework	PyTorch 2.3
Key Libraries	OpenCV and NumPy
Hardware	Raspberry pi 5

2.2 Data

Datasets were just images and not a lot preprocessing steps were needed except fixing the distortion on the images for blindspot and collision warning. The dataset was split into train and test splits but were not needed as the project used a pretrained model, YOLOv8n.

2.3 Algorithm / Approach

Describe your method at a level of detail sufficient for replication. Use pseudocode where it adds clarity:

1. Blindspot detection
 - undistorted
 - yolo model
 - create zones

- detection if a car is in the zone they are in the blind spot
- 2. Collision warning
 - undistorted
 - etc
 - etc
 - etc
 - collision detection
- 3. Lane departure
 - same as collision warning
 - use previous lane results to influence the current lane results, making the lane detection more accurate.

2.4 Experimental Design

The main metric that was measured was the speed that the systems ran on a raspberry pi, as the purpose of the project is to run detections on a system locally that would be attached to a car.

3 Results

3.1 Quantitative Results

results of the systems running on the raspberry pi 5 at some fps.

3.2 Figures and Visualizations

Images of each of the three systems running, and maybe even the smaller steps in the sytem like canny edge, hough, etc.

3.3 Final Product

Current plan of making a website in which you can upload you own set of images

4 Conclusions

1. Summary & Context
 - Goal achieved: Replaced expensive LiDAR/Radar with a vision-only software pipeline (YOLOv8n + CV) on a \$180 Raspberry Pi 5.
 - Final Results: System successfully runs at a stable 5-6 FPS, featuring EMA-stabilized lane tracking and accurate monocular depth estimation.
2. Limitations & “Regrets”
 - Hardware bottleneck: The Pi 5 forced the use of the “nano” YOLOv8 model, reducing long-range detection accuracy.

- Sensor vulnerability: Purely optical systems inherently fail in bad weather (rain, fog, glare).
- Testing limits: Subsystems were validated independently; regret lacking a unified, multi-camera physical vehicle rig for synchronous testing.

3. Future Work

- Compute Upgrade: Migrate to dedicated AI accelerators (Google Coral TPU / NVIDIA Jetson) to achieve 30+ FPS.
- Sensor Fusion: Integrate cheap mmWave radar to overcome camera weather vulnerabilities.
- Vehicle Integration: Interface with the vehicle's CAN-bus to transition from passive warnings to active emergency braking (Level 1 Autonomy).

References